

auto-test-evidence-harness 改訂要件仕様調査報告

エグゼクティブサマリー

`auto-test-evidence-harness` を、単なる「テスト結果の収集器」ではなく、**自動テスト証跡の信頼性を評価し、その評価結果を Quality Evidence Graph に接続して Gate 判定まで一貫させるハarness**として再定義するのが最も妥当です。既存の RNA4219 リポジトリ群はすでに、**手動観点生成 (manual-bb-test-harness)**、**差分とリスクの抽出 (code-to-gate)**、**証跡グラフと DQ 優先ガバナンス (quality-evidence-graph)**、**Evidence 契約 (agent-protocols)**、**実行統制 (shipyard-cp / agent-state-gate / agent-gatefield)** という骨格を持っています。Ateh の役割は、この骨格に対して **自動テスト固有の証跡収集・正規化・評価・可視化・外部エコシステム接続**を埋めることです。 ¹

設計上の最大の論点は、「カバレッジを証跡として扱う」だけでは足りず、**そのカバレッジとテスト自体の信用を評価しなければならない**ことです。これは研究的にも支持されます。コードカバレッジは未テスト領域の発見には有効ですが、テスト有効性の代理指標としては強くなく、変異テストやフレーク制御、要件トレーサビリティ、変更差分への関連性の方が信頼性判断には重要です。したがって ATEH では、Coverage を一指標に留め、**AETE: Auto-Test Evidence Trust Evaluation** を中核に置くべきです。 ²

MVP は広げすぎない方がよく、優先度は **GitHub Actions provenance + JUnit 系 test results + coverage (LCOV/Cobertura/JaCoCo) + SARIF + Playwright trace/screenshot/video + QEG export** です。これだけで多言語・多レイヤーの証跡をかなりの程度統合できます。Pact、Stryker、Allure、OpenTelemetry、ReportPortal、Codecov、SonarQube はその上で段階的に入れるのが実装コストと価値のバランスがよいです。 ³

最終的な推奨像は、**ATEH = CI 上の evidence collector / normalizer / correlator / evaluator / exporter**、**QEG = 判定グラフと DQ ガバナンスの実行系**、**code-to-gate = diff→risk→test-seed 抽出器**、**manual-bb-test-harness = 人間観点の上流生成器**、**agent-state-gate / gatefield = 人間レビューや publish を含む上位統制系**、という責務分離です。これにより、ユーザーが言う「手動テストゼロ」の意味を、**無統制の自動化ではなく、証跡付きで説明可能な自動品質保証**として成立させられます。 ⁴

調査方法と設計前提

本調査では、RNA4219 の公開リポジトリ README / リポジトリページ、各 OSS の公式ドキュメント、ならびに原著論文または原著に近い一次資料を優先して参照しました。特に、Playwright、Vitest、pytest-cov / coverage.py、JaCoCo、Pact、Stryker、SARIF、Allure、ReportPortal、Codecov、SonarQube、OpenTelemetry、GitHub Actions の公式資料を設計根拠として採用しています。 ⁵

RNA4219 側では、`manual-bb-test-harness` が「**根拠付き観点→リスク→優先度→手動テストケース→工数→Gate 判定→Go/No-Go brief**」という出力連鎖を明示し、`code-to-gate` が `findings.json`、`risk-register.yaml`、`test-seeds.json`、`release-readiness.json`、`results.sarif` を生成し、`quality-evidence-graph` が**要求・差分・リスク・テストレイヤー・証跡アーティファクト・Gate**を単一グラフ上で扱い、DQ を waiver では消せないという強い統制を持っています。`agent-protocols` は Evidence / Acceptance / PublishGate 契約を定義し、`workflow-cookbook` は Evidence tracking と release evidence を持ち、`shipyard-cp` は `plan -> dev -> acceptance -> integrate -> publish` を task/run/gate/audit モデルで統制します。 ⁶

一方で、ATEH に直接必要な「自動テスト証跡の正規化」「カバレッジとテストの信用評価」

「diff→risk→test→evidence の可視化」は、既存群では一部が散在しているものの、専用フレームワークとしては未成立です。`portfolio` には `blueprint-to-playwright` と `ci-flaky-analyzer` があり、`test-portfolio-2026` は手動設計中心で Playwright は小規模サンプルに留まり、`Errorcook_Contextpatch` は CI failure を micro-test で再検証し、`agent-gatefield` / `agent-state-gate` は統制判定に強いものの、自動テスト証跡を多様な OSS から吸い上げて信用評価する層は未分離です。これが ATEH の空席です。 ⁷

なお、各 RNA4219 リポジトリの `docs/requirements.md` や `docs/spec/*` など深部文書は、今回の取得範囲では README から確認できた正本参照関係までは検証できましたが、全文の内容までは一律に確認できていません。そのため、**README/公開ページで確認できた事項のみを確定情報とし、それ以上の詳細は unspecified と扱います。** ⁸

調査結果と設計含意

RNA4219 エコシステムから見た ATEH の位置づけ

`manual-bb-test-harness` は、ブラックボックス観点から release gate までの人間系 QA 出力連鎖を定義しています。これは ATEH にとって、**自動テスト証跡が埋めるべき“人間観点の代替/補完対象”を明文化した上流仕様**です。ATEH はこの chain を横取りするのではなく、`root_cause` が diff / risk / test coverage / oracle / flakiness のどこにあるかを、Evidence として自動で埋めるべきです。 ⁹

`code-to-gate` は ATEH の前段として極めて相性がよく、差分・構造・ルール・DB変更を解析して `findings`、`risk-register`、`test-seeds`、`release-readiness`、`SARIF` を吐き出します。これは ATEH が「何を集めるか」を決めるための**変更起点のスコーピング・ルータ**に使えます。すなわち ATEH はテスト全件の羅列ではなく、**`code-to-gate` が抽出した changed/risky paths を中心に evidence を結線**することで、diff→risk→test の説明責任を持てます。 ¹⁰

`quality-evidence-graph` は、ATEH の最終出力先として最も自然です。QEG 側はすでに CLI contract (`validate`, `gate`, `record`) と exit code、DQ 優先、controlled governance、fixture regression を持っているため、ATEH 側で policy / scoring / provenance を整理し、**QEG に「十分に説明可能な evidence bundle」を渡す**構造が実装上も運用上も安定します。ATEH が直接 Go/No-Go を完結させるより、QEG と役割分担する方が再利用性が高いです。 ¹¹

`agent-protocols`、`workflow-cookbook`、`shipyard-cp`、`agent-state-gate`、`agent-gatefield` は、ATEH が後段で publish / human review / approval binding に接続するための基盤です。とくに `agent-state-gate` が approval を `diff_hash` と `context_hash` に束縛して古い承認の再利用を防ぐ点は、ATEH の provenance 設計でもそのまま活かすべきです。ATEH が出す Gate 判定は、単発 JSON ではなく、**後段で再演算・監査できる binding 付き packet** である必要があります。 ¹²

公開 OSS と学術から見た設計含意

公式 OSS 群を見ると、テスト結果・カバレッジ・付随証跡・静的解析結果の**標準的な搬送体**はほぼ出揃っています。Playwright は trace viewer、screenshots、videos、attachments、JUnit-style XML reporter を提供し、pytest-cov / coverage.py は HTML/XML/JSON/Markdown/LCOV に加えて **coverage context** により「どのテストがどの行を通したか」を記録できます。JaCoCo は XML を公式出力し、SonarQube は coverage と execution を外部ツールから import する設計で、SARIF は静的解析結果の共通交換フォーマット、GitHub も SARIF 2.1.0 subset を受け付けます。つまり ATEH が新たに発明すべきなのはフォーマットではなく、**横断正規化と信頼評価**です。 ¹³

Pact、Stryker、Allure、ReportPortal、Codecov、OpenTelemetry は、ATEH を「単なる集計」から「信頼可能な証跡オーケストレータ」へ引き上げる補助線です。Pact は `can-i-deploy` と verification result publication により、単体/結合テストでは見えない**インタフェース整合性と deploy-safe 判定**を与えます。Stryker は mutant states / metrics により、coverage の弱点を補う **oracle strength / adequacy** 指標を与えます。Allure と ReportPortal は attachment・timeline・history・dashboard を提供し、Codecov は PR diff coverage と flags、OpenTelemetry は logs/metrics/traces の相関と semantic conventions を提供します。ATEH はこれらを「置き換える」のではなく、**エビデンス正本を ATEH/QEG に置いたうえで、見せ方や補助分析に外部ツールを使うのがよいです。** ¹⁴

学術的には、ATEH が coverage を“主役”にしてはいけない理由が明確です。Inozemtseva と Holmes は、テストスイートサイズを統制すると coverage と effectiveness の相関は低～中程度であり、強い coverage criterion ほど有効性を保証するわけではないと報告しています。これに対し、Just らは mutant detection と real fault detection の有意な相関を示しており、mutation 系指標は coverage より adequacy 寄りの情報を持ちます。また、Luo らは flaky test が regression testing の信頼を損ない、主因に race condition、external dependency、statefulness が多くを示しました。さらに、Cleland-Huang らの traceability 研究は、要求から下流成果物までのトレースが重要でありながら実務では ad-hoc になりやすいことを示しています。ATEH のスコアリングは、この四点を直に反映すべきです。 ¹⁵

候補 OSS アダプタ比較

次表は、ATEH の一次対象として妥当なアダプタ候補を、**正規化しやすさ・説明可能性・RNA4219 既存資産との接続性**の観点で要約したものです。内容は各公式 docs と RNA4219 公開 README の要約です。 ¹⁶

カテゴリ	候補	強み	弱み	ATEH 優先度
テスト結果	JUnit XML	事実上の共通交換形式。 CI 連携が広い	単一の厳密な公式 universal schema が 弱い	P0
テスト結果	Open Test Reporting	JUnit Platform の新しい event-based 形式	エコシステム浸透は まだ限定的	P1
E2E 証跡	Playwright trace/ video/screenshot	失敗時の再現性・可観測 性が高い	ストレージが重い	P0
カバレッジ	LCOV	JS/TS で普及、差分可視 化に強い	per-test 文脈が弱い	P0
カバレッジ	Cobertura XML	import 先が多い	DTD/互換差異に注意	P0
カバレッジ	JaCoCo XML	JVM 向けに安定、 branch/function richness	JVM 中心	P0
カバレッジ	coverage.py contexts	test→line explainability が強い	Python 寄り	P0
静的解析	SARIF	標準交換形式、GitHub code scanning 直結	tool ごとの差異吸収 が必要	P0
契約	Pact + Broker	can-i-deploy で deploy safe 判定が可能	契約試験未導入組織 では初期コスト	P1

カテゴリ	候補	強み	弱み	ATEH 優先度
Adequacy	Stryker MTE	mutation score でテストの強さを見られる	実行コスト高	P1
可視化	Allure	attachments / history / timeline が強い	本体の verdict graph は持たない	P1
可視化	ReportPortal	履歴・分析・dashboard に強い	サーバ運用負荷	P2
PR 分析	Codecov	diff / flags / PR comment が強い	正本としては外部依存が強い	P2
品質ポータル	SonarQube	coverage/execution import と quality gate	テスト証跡そのものの正本ではない	P2
観測性	OpenTelemetry	traces/logs/metrics 相関が可能	test artifact 自体は別管理が要る	P1

別の観点として、可視化/エクスポート先は「どこを正本にするか」で明確に分けるべきです。ATEH/QEG を正本にし、Allure/ReportPortal/Codecov/SonarQube は二次利用先に留める方が、一貫した DQ と AETE を維持できます。 ¹⁷

出力先	向いている用途	ATEH との関係
QEG	判定正本、DQ、監査	正本
GitHub Job Summary / Artifact	CI 即時可読性、配布	一次配布
Allure	人間が失敗経緯を追う	二次可視化
ReportPortal	長期履歴・チーム分析	二次分析
Codecov	PR coverage review	二次通知
SonarQube	継続的コード品質統合	二次品質ポータル
OpenTelemetry backend	CI observability / troubleshooting	実行観測

改訂要件仕様

製品定義

`auto-test-evidence-harness` は、CI 実行から得られる自動テスト関連シグナルを収集し、正規化し、diff/risk/test/evidence の因果関係に沿って相関し、AETE によって信頼評価し、QEG や外部可視化ツールへ輸出する **local-first / CI-first の evidence harness** と定義するのがよいです。QEG が既に要求・差分・リスク・証跡・Gate を結ぶ runtime である以上、ATEH はそこで不足する「自動テスト証跡の接続装置」であるべきです。 ¹⁸

改訂機能要件

以下を改訂後 FR の推奨セットとします。ここでは“実装すべき最低粒度”まで落としています。

ID	機能要件
FR-01	GitHub Actions / CI 実行から provenance を収集する
FR-02	JUnit XML / OTR / Playwright / pytest / Vitest / Jest / JUnit 実行結果を canonical test result に正規化する
FR-03	LCOV / Cobertura XML / JaCoCo XML / coverage.py context を canonical coverage に正規化する
FR-04	SARIF、Pact verification、Stryker mutation result を canonical evidence として取り込む
FR-05	Playwright trace、screenshot、video、Allure attachment 等を artifact manifest で一元管理する
FR-06	code-to-gate の findings / risk-register / test-seeds / release-readiness を入力に受ける
FR-07	manual-bb-test-harness の観点/リスク/優先度モデルに対応できる test obligation モデルを持つ
FR-08	diff→risk→test→evidence の相関グラフを構築する
FR-09	AETE を evidence item、test case、suite、PR/change 単位で算出する
FR-10	DQ ルールを評価し、失格証跡を gate 計算から除外または全体失格にする
FR-11	QEG へ node/edge bundle として export する
FR-12	GitHub Job Summary、Markdown、HTML、JSON、SARIF、OTel event を出力する
FR-13	Allure / ReportPortal / Codecov / SonarQube への export adapter を持つ
FR-14	record 相当の監査可能な own-output validation record を生成する
FR-15	agent-protocols Evidence / Acceptance / PublishGate 契約へ写像できる

これらはRNA4219エコシステムの既存責務分離に素直に乗ります。code-to-gate が“何を試験すべきか”を、ATEH が“何が実際に証明できたか”を、QEG が“その証明で通してよいか”を担当する構図です。 19

非機能要件

非機能面では、単なるレポート生成ツールではなく証跡系インフラとしての要件が必要です。 20

ID	非機能要件
NR-01	Local-first で、外部 SaaS がなくても評価完結できること
NR-02	すべての normalized artifact に schema version を持たせること
NR-03	commit SHA / run_id / attempt / timestamp / artifact hash を必須 provenance とすること
NR-04	同一入力に対する gate / score は決定的であること
NR-05	アダプタ失敗と gate failure を exit code で分離すること
NR-06	シークレットや機密パスは mask/redact policy を通すこと
NR-07	実行ログ・計測値は OpenTelemetry で外部 backend へ送れること

ID	非機能要件
NR-08	大規模 mono-repo を想定し、changed-scope / flags / partial-run に対応すること
NR-09	JSON / NDJSON を第一級とし、下流再計算を可能にすること
NR-10	人間向け summary は Markdown first、機械向けは canonical JSON first とすること

出力物要件

ATEH の成果物は、後段で再利用しやすいように「正規化済みレコード」「相関結果」「評価結果」「可視化素材」に分けるべきです。

出力名	役割
ateh-run.json	CI provenance / repo / sha / matrix / environment
ateh-test-results.ndjson	正規化済み test result
ateh-coverage.ndjson	正規化済み coverage slice
ateh-static.sarif	正規化または集約済み static result
ateh-contract.ndjson	Pact verification / deploy-safety result
ateh-mutation.ndjson	mutation adequacy result
artifact-manifest.json	trace / screenshot / video / html / log の digest 管理
diff-risk-test.json	diff→risk→test obligation / execution 対応表
risk-coverage-matrix.json	リスク×テスト層×実行有無×AETE
evidence-map.json	QEG export 前の graph bundle
aete-score.json	各 evidence/test/suite/change の信頼スコア
gate-decision.json	verdict / DQ / thresholds / reasons
summary.md	人間向け run summary
summary.html	static HTML summary
qeg-bundle.json	QEG import 用 bundle
otel-events.json	OTel export するイベント / span 素材

CLI 契約

QEG がすでに `validate / gate / record` を採用しているため、ATEH もこれと揃えるのが運用上きれいです。 ²¹

```

ateh collect      # CI/runner outputs を収集
ateh normalize    # test/coverage/static/contract/mutation を canonical 化
ateh correlate    # diff→risk→test→evidence を結線
ateh evaluate     # AETE 計算
ateh gate         # policy + DQ による判定

```

```

ateh report      # md/html/job-summary 出力
ateh export      # qeg/allure/reportportal/codecov/sonarqube/otel
ateh record      # self-validation / audit record 出力
ateh doctor      # adapter / schema / path / provenance チェック

```

推奨 exit code は以下です。これは QEG の現在ルールを踏襲するのが自然です。 ²²

exit code	意味
0	go
1	CLI / schema / adapter failure
2	conditional_go / no_go / disqualified

policy と DQ 指針

Policy は「網羅率の閾値」だけではなく、**証拠が gate 計算に参加できる条件**を定義する必要があります。QEG が DQ を waiver で消せない方針を既に持つため、ATEH も同型でよいです。 ²³

推奨 policy 断面は次の通りです。

```

version: ateh/v1
provenance:
  require_sha_match: true
  require_run_id: true
  require_artifact_hash: true
freshness:
  max_age_hours: 24
coverage:
  changed_lines_min: 0.80
  high_risk_paths_min: 0.90
mutation:
  high_risk_min_score: 0.60
contracts:
  require_can_i_deploy_for_changed_contracts: true
static:
  block_changed_high_or_critical: true
aete:
  min_change_score: 75
  min_high_risk_test_score: 80
dq:
  enabled:
    - ATEH-DQ-01
    - ATEH-DQ-02
    - ATEH-DQ-03

```

推奨 DQ は以下です。既存 QEG の DQ-01..17 とは別 namespace にして、後で cross-map できるようにするのが安全です。 ²³

DQ ID	条件
ATEH-DQ-01	commit SHA が不一致、または欠落
ATEH-DQ-02	schema invalid / parse failed
ATEH-DQ-03	artifact hash 不在または参照ファイル欠落
ATEH-DQ-04	timestamp が run window 外
ATEH-DQ-05	flaky が policy 閾値超過、または unresolved
ATEH-DQ-06	coverage が source file に解決できない
ATEH-DQ-07	changed high-risk path に対応テストが 0
ATEH-DQ-08	coverage だけあり execution result がない
ATEH-DQ-09	Pact <code>can-i-deploy</code> が negative
ATEH-DQ-10	changed path に critical/high SARIF unresolved
ATEH-DQ-11	mutation required path で mutation evidence 欠落
ATEH-DQ-12	browser / platform matrix 必須なのに不足
ATEH-DQ-13	manual override に approval binding 不足
ATEH-DQ-14	report が stale branch / stale baseline に基づく
ATEH-DQ-15	own-output validation の record が生成されない

AETE モデル

AETE の設計原則

AETE は、coverage を中心に据えず、**provenance**、**determinism**、**traceability**、**oracle strength**、**change relevance** を主軸に置くべきです。これは coverage の説明力が限定的である一方、mutation、flaky control、traceability が test trust に直接影響するためです。 ²⁴

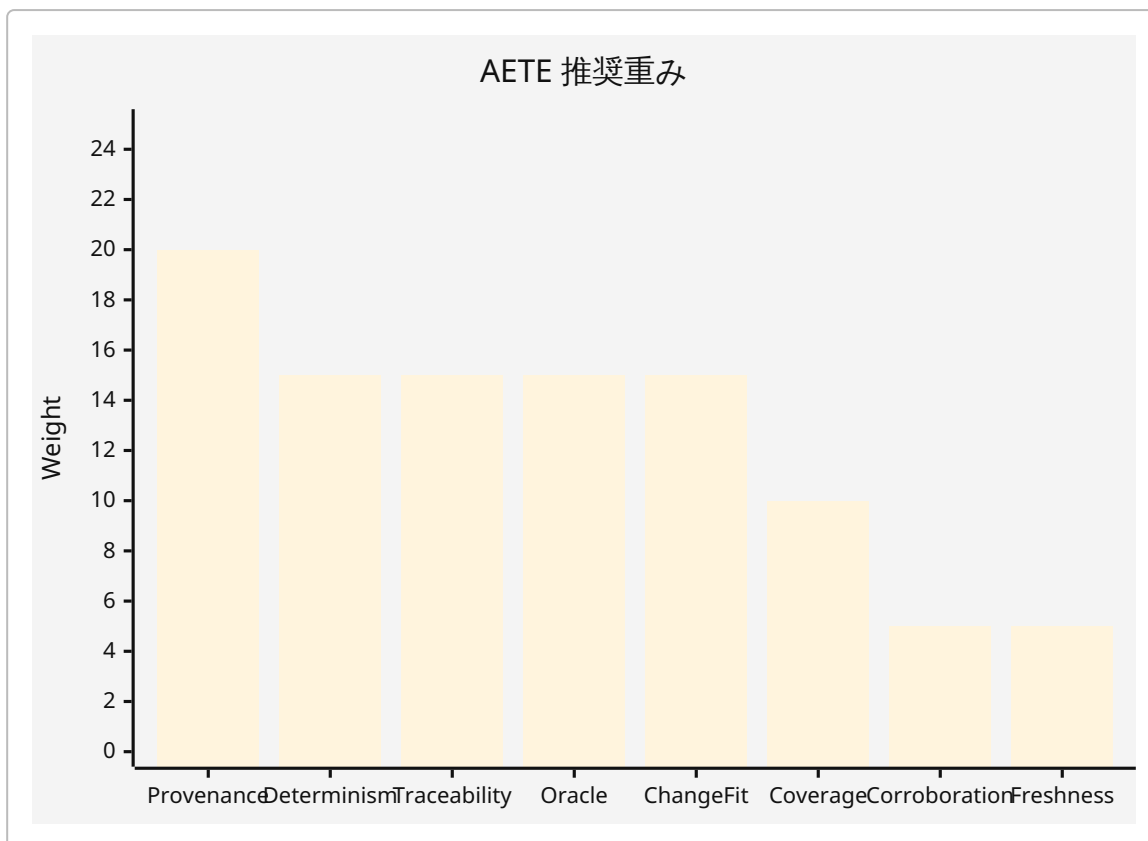
定義は次のように置くと扱いやすいです。

$$AETE(e) = \sum_i w_i \times \frac{s_i(e)}{5}$$

ここで `e` は evidence item または aggregated target、`s_i` は 0-5 の dimension score、`w_i` は重み、合計 100 点です。suite / PR / change 単位のスコアは、risk-weighted aggregation で求めます。

AETE 推奨重み

次の配分が、実務と研究の両面からバランスがよいと考えられます。coverage を 10 に抑えているのがポイントです。 ²⁵



次元	重み	評価観点
Provenance / Integrity	20	SHA, run_id, artifact hash, origin, tamper resistance
Determinism / Flakiness	15	retry, nondeterminism, external dependency control
Traceability / Lineage	15	requirement/risk/diff/test/evidence 連結
Oracle Strength / Adequacy	15	assertion quality, mutation, contract verification
Change Relevance / Risk Fit	15	changed files / risky paths に対する直接性
Coverage Adequacy	10	changed line, branch, risk path coverage
Cross-signal Corroboration	5	execution + coverage + attachment + static の独立裏取り
Freshness / Policy Conformance	5	stale でないこと、policy 適合

AETE 採点規則

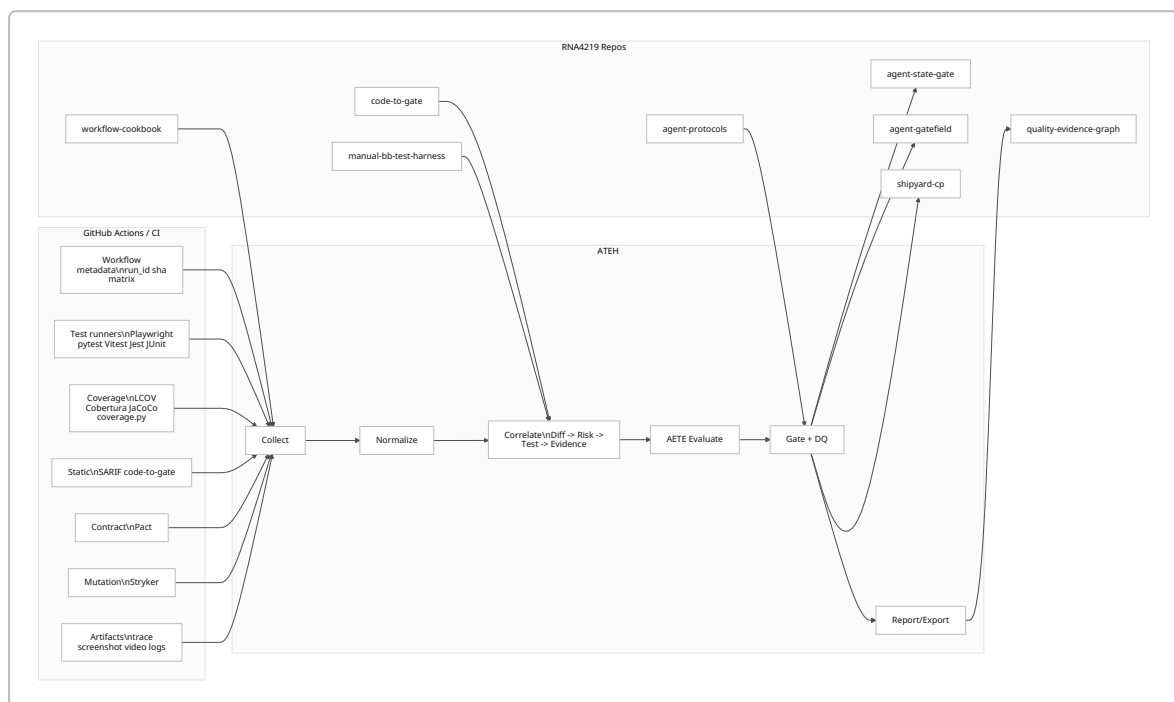
AETE は evidence item ごとに 0-5 を付けてから上位集約します。例として、Playwright failed test で trace/video/screenshot があり、SHA と run_id が一致し、retry で安定再現し、changed path に直結し、Pact と SARIF も矛盾しないなら高得点です。逆に、coverage.xml だけがあり execution result も artifact hash もない場合は DQ か、せいぜい低得点です。 ²⁶

推奨 threshold は以下です。

スコア	意味
90-100	強い自動証拠。監査耐性あり
75-89	実用十分。高リスク変更では補助証拠推奨
60-74	条件付き採用。人手レビューまたは追加試験が必要
0-59	信頼不足。gate 計算で block/hold 相当
DQ	計算対象外または全体失格

アーキテクチャとアダプタ仕様

推奨アーキテクチャ



この構成のポイントは、**収集と判定を分けること**、そして **ATEH 自身が最終統制者にならず QEG・state-gate・gatefield に接続すること**です。shipyard-cp と agent-state-gate が既に orchestration / verdict transformation / audit packet の責務を持つため、ATEH は evidence quality の専門層に留める方が整合的です。 27

入出力 canonical schema の推奨

以下は MVP で先に固定すべき canonical schema です。詳細 JSON Schema は後続実装で分離して構いませんが、フィールドは早期に安定化させるべきです。

```
{
  "ateh-run": {
    "schema_version": "ateh/v1",
    "repo": "owner/name",
    "commit_sha": "abc123",
```

```

"base_sha": "def456",
"ci": {
  "provider": "github_actions",
  "run_id": "123456789",
  "run_attempt": 1,
  "workflow": "ci.yml",
  "job": "test"
},
"env": {
  "os": "ubuntu-latest",
  "language": ["python", "typescript"],
  "matrix": {"python": "3.11"}
},
"started_at": "2026-06-27T01:23:45Z",
"finished_at": "2026-06-27T01:30:12Z"
}
}

```

```

{
  "ateh-test-result": {
    "test_case_id": "playwright:tests/login.spec.ts::login works",
    "framework": "playwright",
    "status": "passed",
    "duration_ms": 1820,
    "retry_index": 0,
    "flaky": false,
    "file": "tests/login.spec.ts",
    "labels": ["e2e", "auth"],
    "artifacts": [
      {"kind": "trace", "ref": "artifacts/trace.zip", "sha256": "..."},
      {"kind": "screenshot", "ref": "artifacts/failure.png", "sha256": "..."}
    ],
    "provenance_ref": "ateh-run.json"
  }
}

```

```

{
  "ateh-coverage-slice": {
    "format": "lcov",
    "file": "src/auth.ts",
    "line_hits": {"10": 1, "11": 1, "20": 0},
    "branch_hits": [{"line": 10, "block": 0, "branch": 1, "taken": 1}],
    "contexts": [
      {"test_case_id": "pytest:tests/test_auth.py::test_login"}
    ],
    "commit_sha": "abc123"
  }
}

```

```
{
  "ateh-evidence-ref": {
    "evidence_id": "EV-AUTO-0001",
    "kind": "execution|coverage|static|contract|mutation|artifact",
    "source_tool": "playwright|pytest-cov|jacoco|sarif|pact|stryker",
    "path": "relative/or/uri",
    "sha256": "...",
    "commit_sha": "abc123",
    "run_id": "123456789"
  }
}
```

```
{
  "ateh-gate-decision": {
    "decision": "go|conditional_go|no_go|disqualified",
    "aete_change_score": 82,
    "dq_hits": [],
    "reasons": [
      "changed high-risk path has direct execution evidence",
      "coverage contexts explain changed lines",
      "no unresolved high severity SARIF in changed files"
    ],
    "policy_version": "ateh/v1",
    "qeg_export_ref": "qeg-bundle.json"
  }
}
```

これらの設計は、GitHub Actions の `GITHUB_RUN_ID` / `GITHUB_RUN_ATTEMPT` / contexts、Playwright attachments、coverage.py contexts、SARIF artifact metadata、OpenTelemetry correlation と自然に整合します。 ²⁸

ATEH 機能と OSS コンポーネントの写像

次表は、ATEH の主要機能をどの OSS に載せるべきかの推奨写像です。各 OSS の公式 docs と RNA4219 既存資産に基づく要約です。 ²⁹

ATEH 機能	主担当コンポーネント	補助コンポーネント
diff/risk/test obligation 生成	<code>code-to-gate</code>	<code>manual-bb-test-harness</code>
execution result ingest	JUnit XML / Playwright reporter / pytest	Vitest / Jest / JUnit OTR
coverage ingest	LCOV / Cobertura XML / JaCoCo XML / coverage.py contexts	Codecov / SonarQube import
static result ingest	SARIF	GitHub Code Scanning
contract evidence	Pact Broker / verifier	<code>can-i-deploy</code>

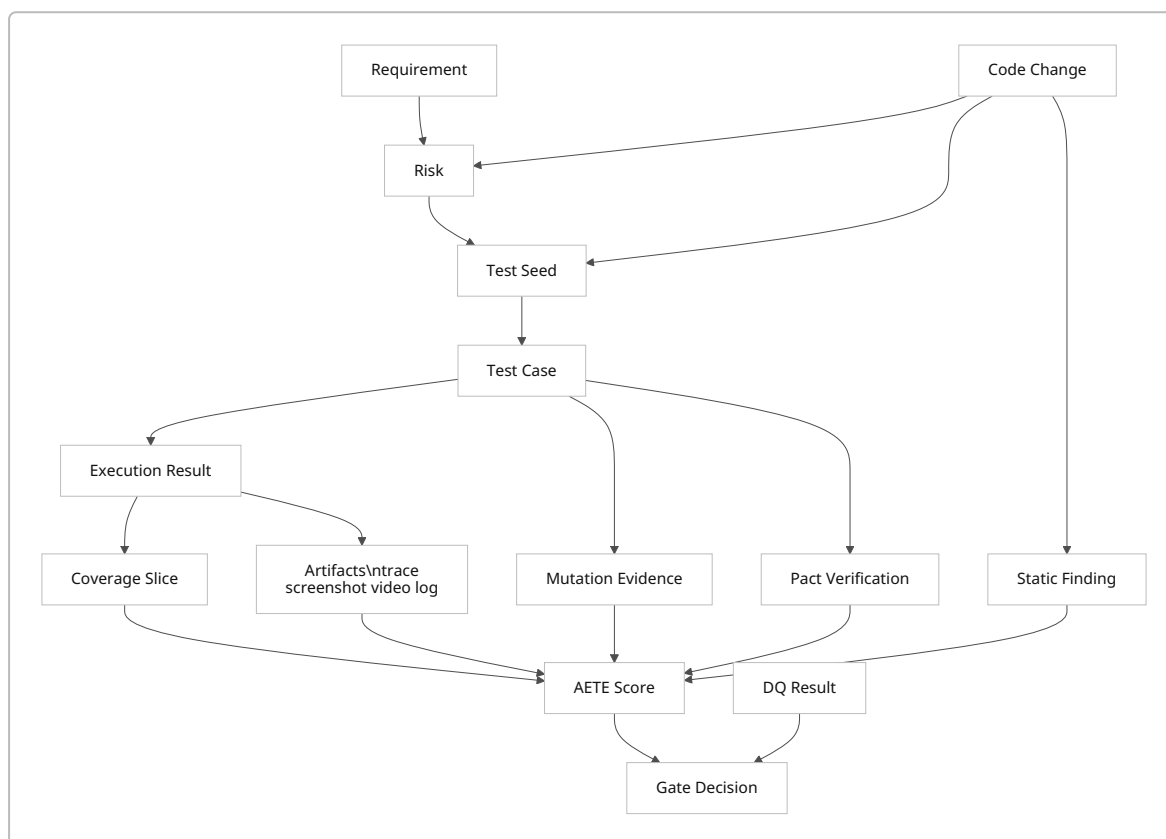
ATEH 機能	主担当コンポーネント	補助コンポーネント
adequacy evidence	Stryker	mutation-testing-elements
artifact UX	Allure	ReportPortal
CI provenance	GitHub Actions metadata / artifacts	OpenTelemetry
verdict graph export	QEG	agent-protocols
post-gate orchestration	agent-state-gate / agent-gatefield / shipyard-cp	workflow-cookbook

requirements → artifacts → QEG nodes の写像

QEG は requirements / code changes / risks / test layers / evidence artifacts / gates を結ぶモデルを持つため、ATEH ではノードの粒度を固定することが重要です。 ³⁰

要件	ATEH artifact	想定 QEG node / edge
FR-01 provenance 収集	ateh-run.json	Run , Environment , generated_by
FR-02 test result 正規化	ateh-test-results.ndjson	TestCase , executed_in , produced_result
FR-03 coverage 正規化	ateh-coverage.ndjson	CoverageSlice , covers , explained_by
FR-04 static / contract / mutation	ateh-static.sarif , ateh-contract.ndjson , ateh-mutation.ndjson	StaticFinding , ContractCheck , MutationCheck
FR-05 artifact manifest	artifact-manifest.json	EvidenceArtifact , attached_to
FR-06 ctg ingest	diff-risk-test.json	CodeChange , Risk , TestSeed
FR-08 correlation	evidence-map.json	targets , mitigates , verifies , evidences
FR-09 AETE	aete-score.json	TrustScore , scored_for
FR-10 gate / DQ	gate-decision.json	GateDecision , disqualified_by
FR-11 QEG export	qeg-bundle.json	import bundle 全体
FR-14 own-output validation	record.json	AuditRecord , validated_by

証跡グラフの推奨構造



このグラフの実務的価値は、「テストが通った」ではなく、**どの要求/差分/リスクを、どのテストが、どの証跡で、どれだけ信頼できる形で支持しているか**を可視化できる点にあります。これは traceability 研究の要請にも合致します。 ³¹

可視化要件と MVP ロードマップ

可視化要件

ATEH の可視化は少なくとも三枚必要です。

第一に **Evidence Map** です。これは requirement / diff / risk / test seed / test case / execution / coverage / artifact / gate をノードとして表示し、commit、file、risk category、test layer、status、AETE score でフィルタできる必要があります。Allure や ReportPortal は個々の実行理解に優れますが、この種の graph-centric なリスク証跡地図は持たないため、ATEH native view が必要です。 ³²

第二に **Risk Coverage Matrix** です。行を risk / changed entity、列を test layer / contract / mutation / static / manual obligation とし、セルに `planned`、`executed`、`covered`、`trusted`、`dq`、`gap` を持たせるべきです。これにより「coverage は高いのに高リスク差分の contract test がない」「UI test はあるが flaky で trust が低い」といった抜けが一目で見えます。Codecov や SonarQube は coverage 面を見るには有用ですが、risk axis が弱いので ATEH で補う意味があります。 ³³

第三に **Diff → Risk → Test** です。PR の changed lines から `code-to-gate` finding / risk を経由し、実際に実行された test case とその attachment / coverage context / mutation / Pact verification に辿れる drill-down

が必要です。Playwright trace viewer、coverage.py contexts、Pact Broker result publication、SARIF location は、この drill-down を組むための一次材料として十分です。 34

MVP 優先順位

MVP は次の順に組むのが適切です。優先順の根拠は「一次証跡の普遍性」「diff/risk との接続のしやすさ」「現行 RNA4219 資産との整合性」です。 35

優先	範囲	理由
P0	GitHub Actions provenance、JUnit XML、Playwright trace/screenshot/video、LCOV/Cobertura/JaCoCo、SARIF、QEG export	多言語 CI を最短でまとめるため
P1	coverage.py contexts、Vitest/Jest adapters、Pact、Stryker、OpenTelemetry、Allure export	信頼評価と explainability を強化するため
P2	ReportPortal、Codecov、SonarQube、agent-state-gate / gatefield 深統合	チーム運用・長期履歴・上位統制へ広げるため

段階ロードマップ

MVP-1 では、collect → normalize → correlate → gate → qeg export の最短ループを完成させます。ここで必要なのは GitHub metadata adapter、JUnit XML adapter、coverage adapter、SARIF adapter、artifact-manifest、ctg reader、qeg exporter、summary.md、job summary です。これだけで PR などの“証跡付き自動品質保証”は成立します。 36

MVP-2 では、AETE を導入します。coverage.py contexts、Playwright traces、flaky analyzer、Stryker、Pact を追加し、coverage 偏重ではないスコアリングを有効化します。portfolio にある ci-flaky-analyzer や blueprint-to-playwright は、そのまま補助資産として再利用できます。 37

MVP-3 では、可視化と外部接続を広げます。Allure / ReportPortal / Codecov / SonarQube / OpenTelemetry export を整備し、組織内の既存運用へ自然に乗せます。ただしこの時点でも正本は ATEH/QEG から動かさない方がよいです。 38

推奨出典

優先して読むべき資料は次の通りです。リンクは引用から辿れるものを優先しました。

公式ドキュメント

- QEG README と repository page: QEG の責務、CLI、DQ 優先、controlled governance の理解に必須。 39
- code-to-gate repository page: outputs と policy model の理解に必須。 10
- manual-bb-test-harness repository page: 上流 QA chain の理解に必須。 9
- agent-protocols README: Evidence / Acceptance / PublishGate 契約の理解に必須。 40
- Playwright docs: Trace viewer、reporters、attachments、video。 41
- coverage.py / pytest-cov docs: contexts、branch coverage、XML/LCOV 出力。 42
- JaCoCo docs: XML report と JVM coverage import の基礎。 43
- OASIS SARIF 2.1.0 と GitHub docs: static result exchange の正本。 44

- Pact docs: verification result publication と `can-i-deploy`。 45
- Stryker docs: mutant states / metrics。 46
- Allure docs: attachments、timeline、history、CSV/metrics export。 47
- ReportPortal docs: dashboards/history、attachments、API。 48
- Codecov docs: formats、flags、PR comments、status checks。 49
- SonarQube docs: coverage/execution import の位置づけ。 50
- OpenTelemetry docs: signals、traces、context propagation、semantic conventions。 51

学術・原著系

- Inozemtseva & Holmes, Coverage Is Not Strongly Correlated with Test Suite Effectiveness. AETE で coverage を過大評価しない根拠。 52
- Just et al., Are Mutants a Valid Substitute for Real Faults in Software Testing? mutation adequacy を AETE に組み込む根拠。 53
- Papadakis et al., Mutation Testing Advances: An Analysis and Survey. mutation testing の設計全体像。 54
- Luo et al., An Empirical Analysis of Flaky Tests. determinism / flaky control を高重みにする根拠。 55
- Cleland-Huang et al., Software traceability: trends and future directions. diff→risk→test→evidence 可視化の理論的根拠。 31

日本語で補助的に有用な資料

- OpenTelemetry の日本語解説。概念の補助理解に有用。 56
- 企業ブログの Playwright 日本語記事。社内導入説明で補助的に使いやすい。 57
- 日本語 SonarQube 紹介記事。組織内説明時の補助資料。 58

以上を踏まえると、ATEH の改訂要件は、**カバレッジ収集器**ではなく、**自動テスト証跡の信用を算出し、QEG に接続し、diff/risk/test/evidence を可視化する証跡ハーネス**として定義するのが最も一貫しています。Coverage は証跡の一部、AETE が品質保証の中核、QEG が最終統制、これが今回の最終結論です。 59

1 manual-bb-test-harness

https://github.com/RNA4219/manual-bb-test-harness?utm_source=chatgpt.com

2 15 24 25 52 Coverage is not strongly correlated with test suite ...

https://dl.acm.org/doi/10.1145/2568225.2568271?utm_source=chatgpt.com

3 36 Workflow commands for GitHub Actions

https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-commands?utm_source=chatgpt.com

4 10 19 35 GitHub - RNA4219/code-to-gate: A generic, local-first code analysis harness that turns repository signals into evidence-backed quality risks, test seeds, and release-readiness gate inputs. / コードベース、変更差分、静的解析、テスト証跡を解析し、スメル検知・品質リスク・テスト設計 seed・リリース判断材料へ変換する汎用ハーネス。・GitHub

<https://github.com/RNA4219/code-to-gate>

5 13 26 34 41 Trace viewer

https://playwright.dev/docs/trace-viewer?utm_source=chatgpt.com

6 9 GitHub - RNA4219/manual-bb-test-harness: Black-box-first skill suite for manual test design, risk-based prioritization, minimal test case synthesis, and release gate decisions. / ブラックボックス前提で、手動テストの観点出し・リスク優先度付け・最小ケース化・リリースゲート判定まで行う skills・GitHub

<https://github.com/RNA4219/manual-bb-test-harness>

7 37 GitHub - RNA4219/portfolio: QA × SDET × LLM – 再現可能な品質成果を示すポートフォリオ · GitHub

<https://github.com/RNA4219/portfolio>

8 11 18 21 22 23 30 39 GitHub - RNA4219/quality-evidence-graph: Quality Evidence Graph is a QA runtime that connects requirements, code changes, risks, test layers, evidence artifacts, and release gates into a reproducible graph for risk-based test placement and Go/No-Go decisions. - Quality Evidence Graph は、要求・コード差分・リスク・テストレイヤー・証跡 artifact・リリースゲートを単一のグラフに接続し、リスクベースのテスト配置と Go/No-Go 判断を再現可能にする QA Runtime です。 · GitHub

<https://github.com/RNA4219/quality-evidence-graph>

12 40 raw.githubusercontent.com

<https://raw.githubusercontent.com/RNA4219/agent-protocols/main/README.md>

14 45 Can I Deploy

https://docs.pact.io/pact_broker/can_i_deploy?utm_source=chatgpt.com

16 29 Reporters

https://playwright.dev/docs/test-reporters?utm_source=chatgpt.com

17 32 38 47 Allure Report Documentation

https://allurereport.org/docs/?utm_source=chatgpt.com

20 Store and share data with workflow artifacts

https://docs.github.com/en/actions/tutorials/store-and-share-data?utm_source=chatgpt.com

27 GitHub - RNA4219/shipyard-cp: AIワーカー群を統治し、Dev / Acceptance / Publish を責務分離して流す self-hosted control plane. · GitHub

<https://github.com/RNA4219/shipyard-cp>

28 Variables reference

https://docs.github.com/en/actions/reference/workflows-and-actions/variables?utm_source=chatgpt.com

31 Software traceability: trends and future directions

https://dl.acm.org/doi/10.1145/2593882.2593891?utm_source=chatgpt.com

33 Flags - Codecov

https://docs.codecov.com/docs/flags?utm_source=chatgpt.com

42 Measurement contexts — Coverage.py 7.14.3 documentation

https://coverage.readthedocs.io/en/latest/contexts.html?utm_source=chatgpt.com

43 JaCoCo - Documentation

https://www.jacoco.org/jacoco/trunk/doc/?utm_source=chatgpt.com

44 Static Analysis Results Interchange Format (SARIF) Version ...

https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html?utm_source=chatgpt.com

46 Mutant states and metrics

https://stryker-mutator.io/docs/mutation-testing-elements/mutant-states-and-metrics/?utm_source=chatgpt.com

48 What is ReportPortal? | ReportPortal Documentation

https://reportportal.io/docs/?utm_source=chatgpt.com

49 Supported Coverage Report Formats - Codecov

https://docs.codecov.com/docs/supported-report-formats?utm_source=chatgpt.com

50 Test coverage parameters | SonarQube Server

https://docs.sonarsource.com/sonarqube-server/analyzing-source-code/test-coverage/test-coverage-parameters?utm_source=chatgpt.com

51 Documentation

https://opentelemetry.io/docs/?utm_source=chatgpt.com

53 Are mutants a valid substitute for real faults in software testing?

https://dl.acm.org/doi/10.1145/2635868.2635929?utm_source=chatgpt.com

54 Mutation Testing Advances: An Analysis and Survey

https://mutationtesting.uni.lu/survey.pdf?utm_source=chatgpt.com

55 An empirical analysis of flaky tests

https://dl.acm.org/doi/10.1145/2635868.2635920?utm_source=chatgpt.com

56 仕様と実装から理解するOpenTelemetryの全体像

https://zenn.dev/yumtdzzz/articles/37c2856f46ea10?utm_source=chatgpt.com

57 Webテスト自動化ツール：Playwrightの魅力と活用法

https://zenn.dev/muit_techblog/articles/e355268058acb7?utm_source=chatgpt.com

58 SonarQube の紹介（と、その日本語化プラグインの紹介）

https://qiita.com/amoriki/items/b512c0ced7b7cb34fef2?utm_source=chatgpt.com

59 Coverage Is Not Strongly Correlated with Test Suite ...

https://cs.ubc.ca/~rtholmes/papers/icse_2014_inozemtseva.pdf?utm_source=chatgpt.com